# Proyecto Integrador 2

Red Social — Modelado y Algoritmos sobre Grafos

Integrantes: Juan González, Gerónimo Lopez y Dalton Crampet

Octubre de 2026

**Descripción general de la solución**

El Proyecto Integrador 2 consistió en construir una biblioteca propia de grafos en Java y utilizarla para resolver situaciones de una red social. El trabajo se organizó en tres desafíos: implementar grafos y algoritmos, modelar usuarios y sus interacciones, y estudiar una operación que pudiera optimizarse. El objetivo fue no solo obtener resultados correctos, sino también comprender las decisiones tomadas y sus costos.

El proyecto utiliza Maven y separa la biblioteca de grafos de las funcionalidades particulares de la red social. Se incorporaron pruebas automatizadas y se compararon dos maneras de obtener los seguidores de todos los usuarios. A continuación se describen las implementaciones, las consultas, los casos especiales y los resultados medidos.

**Desafío 1 — Biblioteca de grafos**

**Estructuras de datos y representación**

La biblioteca implementa las interfaces IGraph e IDirectedIGraph mediante DirectedGraph, y la interfaz IUndirectedGraph mediante UndirectedGraph. Los algoritmos de grafos dirigidos se agrupan en DirectedGraphAlgorithms (IDirectedGraphAlgorithms), mientras que UndirectedGraphAlgorithms implementa IUndirectedGraphAlgorithm. Se respetó así la separación entre la definición del TDA y sus implementaciones concretas.

El grafo de seguimiento es dirigido: una arista A → B significa que A sigue a B, pero B no necesariamente sigue a A. Cada seguimiento utiliza peso 1, de forma que sumar los pesos de un camino equivale a contar sus conexiones. El peso no representa amistad ni intensidad de interacción. La representación se basa en mapas y listas de adyacencias salientes, adecuada para registrar únicamente las relaciones existentes, sin reservar una matriz con todas las parejas posibles.

En los análisis, V representa la cantidad de vértices (usuarios) y E la cantidad de aristas (seguimientos). Para consultar sucesores se revisan las conexiones salientes; obtener predecesores exige buscar las aristas que llegan a un vértice, porque no se guarda un índice permanente de entradas.

**Algoritmos implementados**

**BFS** (búsqueda en amplitud) recorre las conexiones por niveles utilizando una cola. Se aplica, por ejemplo, a los grados de separación: permite encontrar un camino con el menor número de saltos cuando las aristas tienen el mismo peso. DFS (búsqueda en profundidad) avanza por un camino antes de retroceder para explorar otras alternativas; sirve como base de recorridos y análisis de conectividad.

**Dijkstra** obtiene costos mínimos desde un origen cuando los pesos no son negativos. Nuestra implementación guarda costos y predecesores en mapas, registra los vértices procesados en un conjunto y selecciona el menor costo mediante una búsqueda lineal: no utiliza cola de prioridad. **Floyd** calcula costos mínimos entre todos los pares de vértices y **Warshall** determina la alcanzabilidad, utilizando matrices y recorridos sobre vértices intermedios.

La clasificación topológica ordena los vértices de un grafo dirigido sin ciclos, respetando sus dependencias; se implementó con grados de entrada y una cola**. Prim y Kruskal** permiten obtener árboles de costo mínimo en grafos no dirigidos ponderados. Prim usa una cola de prioridad de aristas y Kruskal las ordena por peso y controla los conjuntos de vértices. Si el grafo está desconectado, las implementaciones producen un bosque en lugar de un único árbol.

También se incluyen la detección de ciclos, los puntos de articulación para identificar vértices cuya eliminación desconecta partes de un grafo no dirigido y la enumeración de todos los caminos simples entre dos vértices, evitando repetir un vértice dentro del mismo camino. La excentricidad implementada es de entrada: el mayor costo mínimo necesario para llegar a un vértice desde otros orígenes; el centro es el de menor excentricidad de entrada, cuando existe.

**Casos especiales y pruebas**

Se verificaron escenarios como grafos vacíos, un único vértice, vértices aislados, componentes desconectadas, ciclos, pesos iguales, caminos alternativos y entradas inexistentes. No todas las operaciones resuelven los casos especiales de la misma manera: algunos recorridos lanzan una excepción ante un origen inexistente, los caminos simples entre extremos inexistentes devuelven una lista vacía y el centro puede ser nulo si no existe un candidato alcanzable desde todos los demás.

**Desafío 2 — Modelado de la red social**

**Usuarios, seguidores y seguidos**

La clase RedSocial concentra las operaciones del dominio. registrarUsuario() crea una cuenta y la incorpora al grafo, seguir() agrega la relación dirigida y dejarDeSeguir() la quita. seguidos() consulta las conexiones salientes; seguidores() obtiene las entrantes. sonMutuos() comprueba si dos usuarios se siguen entre sí y seguidoresMutuos() devuelve el conjunto de relaciones recíprocas de una cuenta.

Por ejemplo, si Juan sigue a Gero pero Gero no sigue a Juan, Juan aparece entre los seguidores de Gero, aunque la relación no es mutua. El sistema evita duplicar usuarios y seguimientos y prohíbe que una cuenta se siga a sí misma.

**Sugerencias y grados de separación**

La clase SocialGraphQueries implementa sugerencias() y gradosDeSeparacion(). La primera observa a quiénes siguen las cuentas que una persona ya sigue, excluye al usuario original y a sus contactos directos, y ordena los candidatos según la cantidad de conexiones compartidas. Ante un empate se respeta el orden de registro de los vértices.

gradosDeSeparacion() aplica BFS sobre las relaciones dirigidas para calcular el menor número de saltos entre dos cuentas. Por ejemplo, si Juan sigue a Gero y Gero sigue a Dalton, la distancia dirigida de Juan a Dalton es 2; la distancia inversa puede no existir. En ese caso, el método devuelve -1.

**Influencia, comunidades y usuarios puente**

SocialGraphAnalytics incorpora dos criterios de influencia. rankingPorSeguidores() ordena usuarios por cantidad de seguidores directos. rankingPorAlcance() los ordena por cuántas cuentas podrían recibir contenido a través de cadenas de seguidores: para esta medida se recorren las conexiones en dirección inversa a la arista «A sigue a B». Se trata de alcance estructural potencial, distinto del alcance de un posteo real, que depende de menciones y compartidos.

comunidades() identifica componentes fuertemente conexas: grupos en los que existe un camino dirigido de ida y vuelta entre cualquier pareja de usuarios del mismo grupo. usuariosPuente() crea una proyección no dirigida del seguimiento y busca puntos de articulación. Así detecta cuentas cuya eliminación aumentaría la cantidad de componentes de esa proyección, sin afirmar que necesariamente exista comunicación dirigida en ambos sentidos.

**Publicaciones, comentarios, compartidos y menciones**

Las entidades Posteo, Comentario y Compartido se almacenan fuera del grafo de seguimientos y mantienen referencias a los usuarios participantes; comentarios y compartidos identifican su posteo por id. RedSocial implementa publicar(), comentar(), compartir(), feed() y alcance(). Las menciones @usuario pueden aparecer tanto en posteos como en comentarios y permiten que una cuenta reciba contenido aunque no siga al autor.

El feed presenta una entrada por publicación, excluye las propias y conserva el motivo de llegada más reciente: publicación de alguien seguido, compartido de alguien seguido o mención. Se ordena con un contador de eventos, no con la fecha del reloj de la computadora. El método Difusion.alcance() reúne destinatarios en un conjunto para evitar duplicados y recorre los seguidores de los usuarios que efectivamente compartieron.

Por ejemplo, si Ana menciona a Bruno, Bruno recibe el posteo aunque no siga a Ana. Si Bruno luego lo comparte, el contenido puede llegar a quienes siguen a Bruno. Recibir el contenido, por sí solo, no implica reenviarlo. El autor tampoco se incluye entre los destinatarios de su propio alcance.

Las pruebas cubren la difusión y la coincidencia entre feed y alcance para los escenarios ensayados. La auditoría identificó un caso aún pendiente de validar: después de eliminar a un usuario intermediario en una cadena de compartidos, el feed podría conservar un posteo cuyo alcance ya no incluye al destinatario. Se deja explicitado como limitación detectada por inspección, no como error confirmado mediante una prueba ejecutada.

**Cinco consultas adicionales**

Para cumplir con la exigencia de realizar cinco operaciones propias que utilicen el grafo, se implementó ConsultasAdicionales. Cada consulta construye un índice temporal con conexiones entrantes y salientes; no se limita a mostrar atributos almacenados ni modifica el grafo original.

1. alcanzablesHastaK(): obtiene los usuarios a los que se llega siguiendo como máximo K conexiones. Utiliza BFS con límite de profundidad. Si Juan sigue a Gero y Gero a Dalton, desde Juan con K = 2 se obtiene {Gero, Dalton}, sin incluir al propio Juan.

2. seguidoresComunes(): compara los conjuntos de seguidores de dos cuentas. Si Juan y Ana siguen tanto a Gero como a Dalton, los seguidores comunes de Gero y Dalton son {Juan, Ana}. Se utiliza la intersección de conjuntos, mediante retainAll().

3. caminoEvitando(): busca mediante BFS un camino dirigido mínimo en cantidad de saltos sin atravesar a una cuenta bloqueada. Si Juan puede llegar a Dalton por Gero o por Ana, al bloquear a Gero podría devolver [Juan, Ana, Dalton]. Si no existe una ruta, devuelve una lista vacía.

4. gruposDeSeguimientoMutuo(): forma grupos conectados utilizando únicamente relaciones recíprocas. Si Juan y Gero se siguen mutuamente y Gero y Dalton también, los tres quedan en el mismo grupo aunque Juan y Dalton no se sigan directamente. Los usuarios sin conexiones mutuas forman grupos unitarios.

5. perdidaPorEliminarSeguimiento(): calcula los alcanzables antes y después de ignorar temporalmente una relación y devuelve la diferencia. Por ejemplo, en Juan → Gero → Dalton → Ana, al simular quitar Gero → Dalton desde Juan se perdería el acceso a {Dalton, Ana}, si no existen caminos alternativos. El grafo no se modifica.

Las cinco consultas tienen costo temporal y espacial O(V + E), porque cada llamada prepara las relaciones del grafo y luego realiza recorridos o comparaciones sobre ellas. Las pruebas específicas se encuentran en ConsultasAdicionalesTest.

**Análisis de complejidad**

La notación O describe cómo crece el trabajo al aumentar el tamaño del grafo. En varios algoritmos se distingue el costo habitual de un recorrido del costo concreto de acceder a las adyacencias, ya que algunas operaciones de la biblioteca buscan linealmente el vértice antes de consultar sus vecinos.

En los recorridos BFS y DFS dirigidos con origen, así como en gradosDeSeparacion(), el costo temporal de esta implementación es $O(V^2 + E)$, debido a las búsquedas adicionales de vértices. El espacio auxiliar habitual es O(V), aunque DFS puede acumular más elementos pendientes.

Dijkstra tiene un costo temporal $O(V^2 + E)$, porque busca linealmente el vértice de menor costo en cada paso; su memoria auxiliar se basa en mapas y conjuntos de tamaño O(V), con materializaciones temporales de aristas. Floyd y Warshall requieren $O(V^3)$ de tiempo y $O(V^2)$ de espacio. El cálculo del centro y la excentricidad utiliza Floyd, por lo que también requiere $O(V^3)$ por llamada.

Para los grafos no dirigidos, Prim utiliza una cola de prioridad de aristas y tiene una cota conservadora $O(V^2 + E \log(E + 1) + V)$. Kruskal ordena aristas y emplea representantes con compresión de caminos, sin unión por rango; una cota conservadora es $O(V + E \log(E + 1) + VE)$. Ambos necesitan O(V + E) de espacio, considerando las estructuras auxiliares y el resultado.

En las consultas sociales, rankingPorSeguidores() cuesta $O(V + E + V \log V)$, mientras que rankingPorAlcance() puede llegar a $O(V(V + E) + V \log V)$, porque calcula el alcance potencial de cada usuario antes de ordenar. Las cinco consultas adicionales cuestan O(V + E) por llamada, tanto en tiempo como en espacio.

Finalmente, la consulta original de seguidores de todos los usuarios cuesta O(V(V + E)), mientras que la variante optimizada cuesta O(V + E). Ambas necesitan O(V + E) de espacio para representar los seguidores obtenidos.

En BFS y DFS el costo habitual con acceso directo a adyacencias sería O(V + E), pero los métodos públicos aquí utilizados añaden búsquedas lineales de vértices. En Prim y Kruskal las cotas anteriores incluyen de forma conservadora el costo de las estructuras utilizadas y de construir el resultado, no solamente la selección u ordenación de aristas. Estas estimaciones consideran accesos esperados de costo constante a mapas y conjuntos.

**Desafío 3 — Optimización**

**Operación identificada y alternativa**

La operación elegida fue obtener los seguidores de todos los usuarios. En la versión original se llamaba a predecessors() para cada vértice; como el grafo guarda principalmente aristas salientes, cada búsqueda volvía a inspeccionar las relaciones. Repetirla V veces tiene un costo O(V(V + E)).

La alternativa implementada en Difusion.seguidoresDeCadaVertice() inicializa una lista de seguidores por vértice y recorre todas las aristas una sola vez. Por cada relación A → B, agrega A a los seguidores de B. Así se evita realizar una búsqueda individual para cada usuario, con un costo O(V + E). El índice se construye durante la consulta, por lo que refleja los seguimientos vigentes sin mantener una segunda estructura permanente sincronizada.

Ambas variantes producen un resultado que requiere O(V + E) de espacio. La alternativa utiliza listas intermedias que, dentro del experimento, se convierten a conjuntos; ese detalle puede afectar el consumo transitorio, pero no cambia el orden espacial. No se midió el consumo de memoria.

**Pruebas y mediciones**

ExperimentoOptimizacionTest permite activar mediciones con -Dexperimento=true. Para cada tamaño de grafo determinista se realizó un calentamiento básico y se midieron cinco ejecuciones de cada variante mediante System.nanoTime(), comprobando la igualdad de los resultados en cada repetición. Los tiempos que se presentan a continuación son promedios de esa ejecución concreta; no representan una garantía universal de rendimiento.

Con 100 usuarios y 400 relaciones, la versión original tardó 4,910 ms y la optimizada 0,882 ms, lo que representa una mejora de 5,57 veces.

Con 300 usuarios y 1.200 relaciones, los tiempos fueron 15,889 ms y 3,361 ms, respectivamente, con una mejora de 4,73 veces.

Con 600 usuarios y 2.400 relaciones, la versión original registró 26,245 ms frente a 4,058 ms de la optimizada, con una mejora de 6,47 veces.

En los tres escenarios la versión optimizada fue más rápida, entre 4,73 y 6,47 veces en las mediciones registradas. Como E = 4V en estos grafos, el costo original crece de manera cuadrática respecto de V, mientras que la alternativa crece linealmente. El resultado confirma la conveniencia de evitar recorridos repetidos para esta operación en los tamaños ensayados.

**Principales decisiones de diseño**

Separamos las estructuras generales de grafos de la lógica propia de la red social para reutilizar algoritmos sin mezclar reglas de publicaciones con recorridos. RedSocial mantiene un mapa usuariosPorId para encontrar cuentas por identificador; las relaciones, en cambio, permanecen en el grafo.

Otra decisión fue ofrecer una vista GrafoSoloLectura mediante grafoDeSeguimiento(), de manera que las altas y bajas pasen por RedSocial y sus validaciones. El sistema rechaza cuentas repetidas y seguimientos a uno mismo, evita duplicar aristas y, al eliminar un usuario, limpia sus relaciones y la actividad asociada. Posteos, comentarios y compartidos se representan por separado para no mezclar publicaciones con los vértices del seguimiento.

Por último, la optimización construye un índice auxiliar de seguidores solo cuando se requiere. Esto evita mantener permanentemente listas de entrada sincronizadas con cada operación de seguir o dejar de seguir, a cambio de volver a crear el índice en cada cálculo.

**Registro de uso de herramientas de inteligencia artificial**

Durante el proyecto se utilizaron herramientas de IA, principalmente ChatGPT y GitHub Copilot, para interpretar la consigna, explicar algoritmos, revisar y generar código, colaborar en la integración, ampliar y revisar pruebas, desarrollar consultas adicionales, preparar y evaluar la optimización y redactar y auditar el informe. También se utilizaron para identificar imprecisiones entre la documentación y la implementación.

Las funcionalidades se contrastaron con el repositorio y se ejecutaron pruebas con Maven; los resultados experimentales consignados provienen de la ejecución realizada para la comparación de seguidores.

**Conclusión**

El proyecto permitió aplicar los grafos a una situación concreta y comprender la importancia de la dirección de los seguimientos, los recorridos, los caminos y la conectividad para responder consultas de una red social. La separación entre biblioteca y dominio facilitó organizar las funcionalidades y reutilizar los algoritmos.

La comparación de seguidores demostró que una operación puede devolver resultados correctos y, sin embargo, realizar trabajo repetido. Al reunir la información en una pasada se redujeron los tiempos de ejecución en los tres escenarios probados, manteniendo los mismos resultados. La revisión final también permitió identificar límites y casos pendientes de comprobación, que deberán considerarse durante la defensa del proyecto.